

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине **Основы программирования систем управления**

Тема лабораторной работы: «Программное обеспечение системы управления мехатронного модуля управления защитной дверью»

Студенты группы: КРБО-01-23 Корнев К. А. _____

Мино П. С. _____

Ярошенко А. В. _____

Преподаватель: ст. преподаватель Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

ЦЕЛЬ РАБОТЫ

Получение навыков построения программного обеспечения промышленных систем управления на базе функциональных блоков и конечных автоматов.

ЗАДАНИЕ

Разработать программное обеспечение системы автоматического управления приводом защитной двери. Схема механизма представлена на рисунке 1. Дверь оснащена асинхронным двигателем и четырьмя датчиками положения (S_0–S_3), которые реагируют на пластину, закрепленную на двери. Открытие и закрытие двери управляется тумблером.

При включении системы управления дверь должна двигаться в заданную сторону на небольшой скорости для определения своего местоположения. В этом режиме индикаторы должны мерцать с частотой 1 Гц. После этого происходит переход в рабочий режим.

В рабочем режиме необходимо обеспечить максимально возможную скорость движения на отрезке s_1s_2 и небольшую скорость на отрезках s_0s_1 и s_2s_3 (для обеспечения безопасности движения). Индикаторы должны показывать местоположение двери.

Система управления дверью должна быть выполнена в виде функционального блока, предполагающего повторное использование.

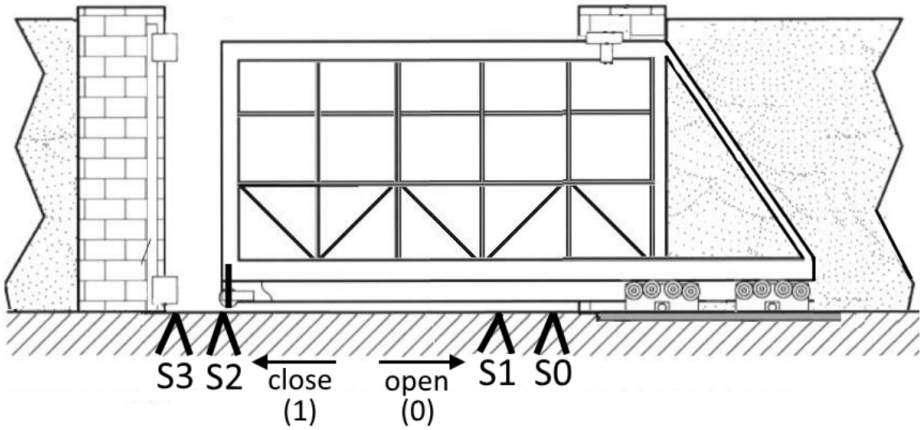


Рис. 1 — Устройство защитной двери

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Открыть исходную конфигурацию оборудования и проинициализировать частотный преобразователь;
    1.1. Запустить B&R Automation Studio;
    1.2. Открыть проект исходной конфигурации оборудования;
    1.3. Открыть визуальную конфигурацию оборудования и добавить к имеющимся модулям частотный преобразователь 8I64xxxxxxx.00x-1;
    1.4. Присоединить модуль частотного преобразователя к шине X2X.
2. Сконфигурировать асинхронный двигатель, используя параметры, указанные на его корпусе;
3. Создать в проекте необходимые объекты;
    3.1. Создать в проекте объекты:
        3.1.1. ANSI C Program;
        3.1.2. ANSI C Library.
    3.2. В библиотеке создать тип данных — перечисление «DoorStates» и функциональные блоки:
        3.2.1. «DriveStateMashine» — функциональный блок управления частотным преобразователем;
        3.2.2. «DoorStateMashine» — функциональный блок, реализующий основную логику программы: задание направления и скорости движения двери в зависимости от ее положения и требуемого направления;
        3.2.3. «LedStateMashine» — машина состояний обработки светодиодных индикаторов.
4. Провести операцию мэпинга индикаторов, датчиков положения и частотного преобразователя;
5. Написать программный код логики управления дверью;
6. Провести эксперимент, подтверждающий работоспособность системы управления.

ХОД РАБОТЫ

Работа выполнялась в среде B&R Automation Studio 4.5 (Automation Runtime D4.52) на учебном стенде. Состав используемого оборудования приведен в таблице 1.

Таблица 1 — Используемое оборудование

Модуль	**Назначение**
4PP065.0571-P74	панельный контроллер (Power Panel), на котором выполняется программа
X20BC0083	шинный контроллер POWERLINK, связывающий контроллер с модулями ввода-вывода
X20DI9371	модуль дискретных входов: тумблер направления и датчики положения S_0–S_3
X20DO9322	модуль дискретных выходов: светодиодные индикаторы положения двери
X20BT9100	модуль шины X2X Link для подключения частотного преобразователя
8I64xxxxxxx.00x-1	частотный преобразователь, управляющий асинхронным двигателем привода двери

В начале работы был открыт проект с исходной конфигурацией оборудования. В визуальной конфигурации оборудования (Physical View) к имеющимся модулям был добавлен частотный преобразователь 8I64xxxxxxx.00x-1, подключенный по шине X2X к модулю X20BT9100 (рисунок 2).

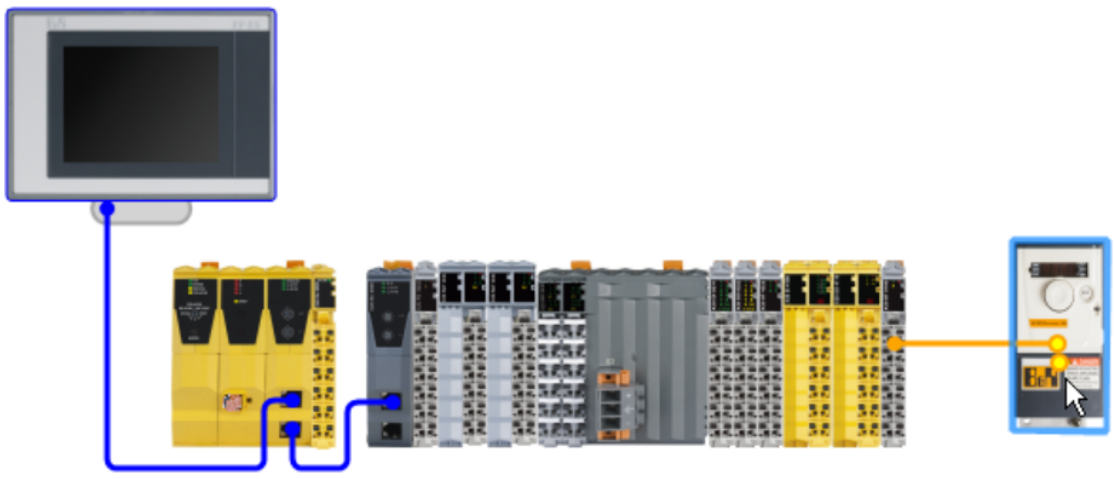


Рис. 2 — Конфигурация оборудования проекта

Затем в конфигурации частотного преобразователя (пункт контекстного

меню «Configuration») были заданы параметры асинхронного двигателя, указанные на его корпусе (таблица 2). Там же для регистров ETAD и CMDD было выбрано представление в виде целых слов (packed), а не отдельных битовых полей. Благодаря этому слово состояния и слово управления преобразователя связываются с одной переменной каждое.

Таблица 2 — Параметры асинхронного двигателя

Параметр	**Обозначение**	**Значение**
номинальное питающее напряжение	UNS [В]	220
номинальная питающая частота	FRS [0,1 Гц]	500
сопротивление статора	RSC [мОм]	33000
коэффициент мощности	COS [0,01]	64
номинальная скорость вращения	NSP [об/мин]	890
номинальный ток	NCR [0,1 А]	10

Далее в проекте (Logical View) были созданы объекты: ANSI C Program «Program» и ANSI C Library «Library». В библиотеке был создан тип данных — перечисление `DoorStates`, содержащее состояния двери, а также три функциональных блока: `DriveStateMashine`, `DoorStateMashine` и `LedStateMashine`. В файле констант библиотеки `Constants.var` были объявлены скорости движения, коды состояний и команды частотного преобразователя. Экземпляры функциональных блоков объявлены как глобальные переменные в файле `Global.var`: `stateMachine`, `doorSM` и `ledSM`.

После этого был выполнен мэпинг переменных, то есть их привязка к каналам модулей ввода-вывода (таблица 3). Благодаря мэпингу программа работает с обычными переменными, а значения каналов передаются в них и из них автоматически в каждом цикле обмена.

Таблица 3 — Мэпинг переменных

Канал модуля	**Переменная**	**Описание**
8I64: ETAD_Input	`stateMachine.state`	слово состояния преобразователя

Канал модуля	**Переменная**	**Описание**
8I64: CMDD_Output	`stateMachine.command`	слово управления преобразователем
8I64: LFRD_Output	`stateMachine.speed`	задание скорости
X20DI9371: DigitalInput01	`doorSM.direction`	тумблер направления
X20DI9371: DigitalInput02	`doorSM.sw1`	датчик S_0 (дверь открыта)
X20DI9371: DigitalInput03	`doorSM.sw2`	датчик S_1
X20DI9371: DigitalInput04	`doorSM.sw3`	датчик S_2
X20DI9371: DigitalInput05	`doorSM.sw4`	датчик S_3 (дверь закрыта)
X20DO9322: DigitalOutput02	`ledSM.led1`	индикатор S_0
X20DO9322: DigitalOutput03	`ledSM.led2`	индикатор S_1
X20DO9322: DigitalOutput04	`ledSM.led3`	индикатор S_2
X20DO9322: DigitalOutput05	`ledSM.led4`	индикатор S_3

Затем был написан программный код логики управления дверью. Проект был собран (Rebuild All) и загружен в контроллер (Transfer).

Логика управления дверью реализована в виде конечного автомата, его структурная схема приведена на рисунке 3. В верхнем ряду схемы расположены состояния движения на закрытие (тумблер в положении `direction = 1`, скорость положительная), в нижнем ряду — состояния движения на открытие (`direction = 0`, скорость отрицательная). Внутри каждого состояния указаны заданная скорость и код индикации (индикаторы $S_0S_1S_2S_3$).

Переходы между состояниями движения выполняются по изменению логического уровня датчика положения (переход $0 \to 1$ или $1 \to 0$), а не по самому значению сигнала. Переходы только последовательные: каждый датчик проверяется лишь в том состоянии, которое заканчивается на этом датчике, поэтому перескочить через состояние нельзя. Вертикальные переходы на схеме соответствуют реверсу: при переключении тумблера во время движения дверь меняет направление на том же отрезке хода. Кроме того, из любого состояния при потере готовности частотного преобразователя (авария или отключение) автомат возвращается в состояние `ST_INIT`, после чего определение положения двери выполняется заново.

Состояния двери заданы перечислением `DoorStates`. Описание состояний, заданные в них скорость и индикация, а также условия перехода приведены в таблице 4. Скорость задается константами `SPEED_SLOW` = 300 и `SPEED_FAST` =

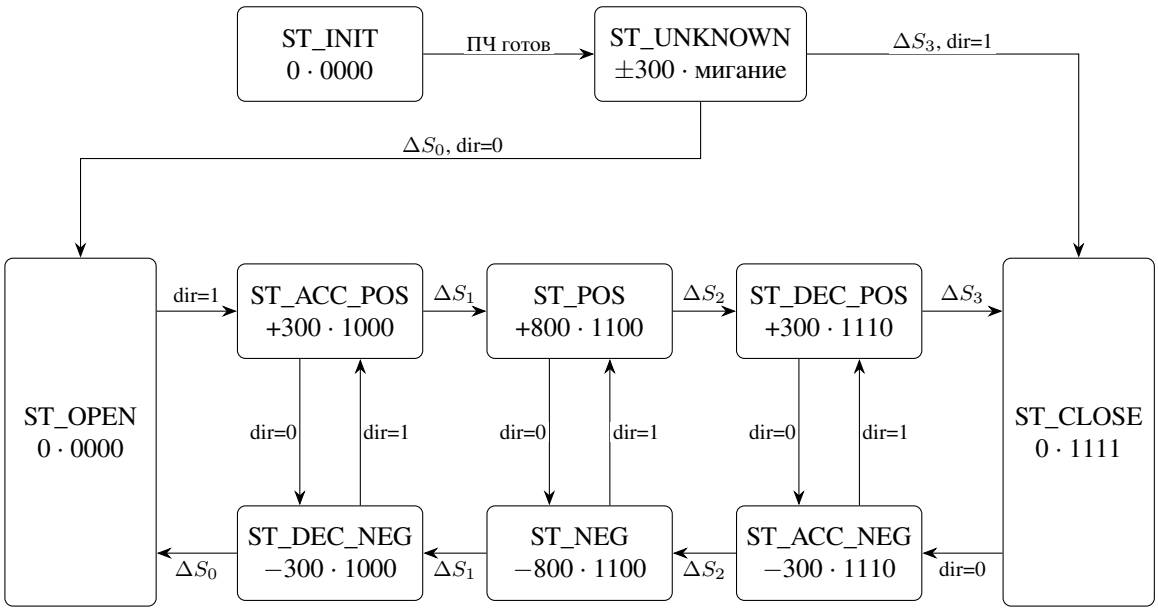


Рис. 3 — Структурная схема конечного автомата управления дверью (ΔS_i — изменение логического уровня датчика S_i)

800, ее знак определяет направление вращения двигателя.

Таблица 4 — Состояния конечного автомата двери

Состояние	**Описание**	**Скорость**	**Индик.**	**Переходы**
`ST_INIT`	инициализация и ожидание включения частотного преобразователя	0	0000	ПЧ в состоянии «работа разрешена» → `ST_UNKNOWN`
`ST_UNKNOWN`	дверь в неизвестном положении, поиск положения на малой скорости в сторону тумблера	±300	мигание 1 Гц	dir=0 и ΔS_0 → `ST_OPEN`; dir=1 и ΔS_3 → `ST_CLOSE`
`ST_OPEN`	дверь открыта (на датчике S_0)	0	0000	dir=1 → `ST_ACC_POS`
`ST_ACC_POS`	разгон на закрытие, отрезок $s_0 s_1$	+300	1000	ΔS_1 → `ST_POS`; dir=0 → `ST_DEC_NEG`
`ST_POS`	движение на закрытие, отрезок $s_1 s_2$	+800	1100	ΔS_2 → `ST_DEC_POS`; dir=0 → `ST_NEG`
`ST_DEC_POS`	замедление на закрытие, отрезок $s_2 s_3$	+300	1110	ΔS_3 → `ST_CLOSE`; dir=0 → `ST_ACC_NEG`
`ST_CLOSE`	дверь закрыта (на датчике S_3)	0	1111	dir=0 → `ST_ACC_NEG`
`ST_ACC_NEG`	разгон на открытие, отрезок $s_3 s_2$	−300	1110	ΔS_2 → `ST_NEG`; dir=1 → `ST_DEC_POS`

Состояние	**Описание**	**Скорость**	**Индик.**	**Переходы**
ST_NEG	движение на открытие, отрезок $s_2 s_1$	-800	1100	$\Delta S_1 \to$ ST_DEC_NEG; dir=1 $\to$ ST_POS
ST_DEC_NEG	замедление на открытие, отрезок $s_1 s_0$	-300	1000	$\Delta S_0 \to$ ST_OPEN; dir=1 $\to$ ST_ACC_POS

Индикация показывает местоположение двери: при движении на закрытие индикаторы загораются по одному (1000, 1100, 1110, 1111), при движении на открытие гаснут в обратном порядке (1110, 1100, 1000, 0000).

Отдельного нарастания скорости в состояниях разгона программа не выполняет: она сразу задает требуемое значение, а плавное изменение частоты вращения обеспечивает сам частотный преобразователь за счет заданных в его конфигурации времен разгона и торможения (параметры ACC и DEC).

Далее были разработаны функциональные блоки библиотеки. Функциональный блок DriveStateMashine выполняет включение частотного преобразователя и передачу ему задания скорости. Входы и выходы блока приведены в таблице 5.

Таблица 5 — Параметры функционального блока DriveStateMashine

Конфигурация	**Имя**	**Тип**	**Описание**
вход	state	UINT	слово состояния частотного преобразователя (ETAD)
вход	enable	BOOL	сигнал работы функционального блока
вход	speedSet	INT	требуемая скорость, рассчитанная логикой двери
выход	command	UINT	команда, подаваемая на частотный преобразователь (CMDD)
выход	speed	INT	заданная скорость (LFRD)

Блок реализует последовательность включения привода по профилю CiA 402 (DRIVECOM), приведенную в методических указаниях. Слово состояния преобразователя является набором битовых флагов (таблица 6). Кроме приведенных битов, в нем есть флаги предупреждения, удаленного управления и достижения задания, которые изменяются независимо от состояния привода. Поэтому перед сравнением на слово накладывается маска 0x6F, оставляющая только биты, определяющие состояние.

Таблица 6 — Биты слова состояния частотного преобразователя

Бит	**Значение**
0	готов к включению
1	включен
2	работа разрешена
3	авария
4	напряжение подано (не входит в маску)
5	быстрый останов неактивен
6	включение запрещено

Схема работы блока приведена на рисунке 4, используемые коды состояний и команд — в таблице 7. В каждом цикле блок определяет текущее состояние преобразователя и подает команду, переводящую его в следующее состояние. Если вход `enable` равен нулю, подается команда `CMD_SHUTDOWN` и скорость обнуляется. Если в слове состояния установлен бит аварии, подается команда сброса ошибки. Задание скорости `speedSet` передается на выход `speed` только в состоянии «работа разрешена», в остальных состояниях скорость равна нулю.

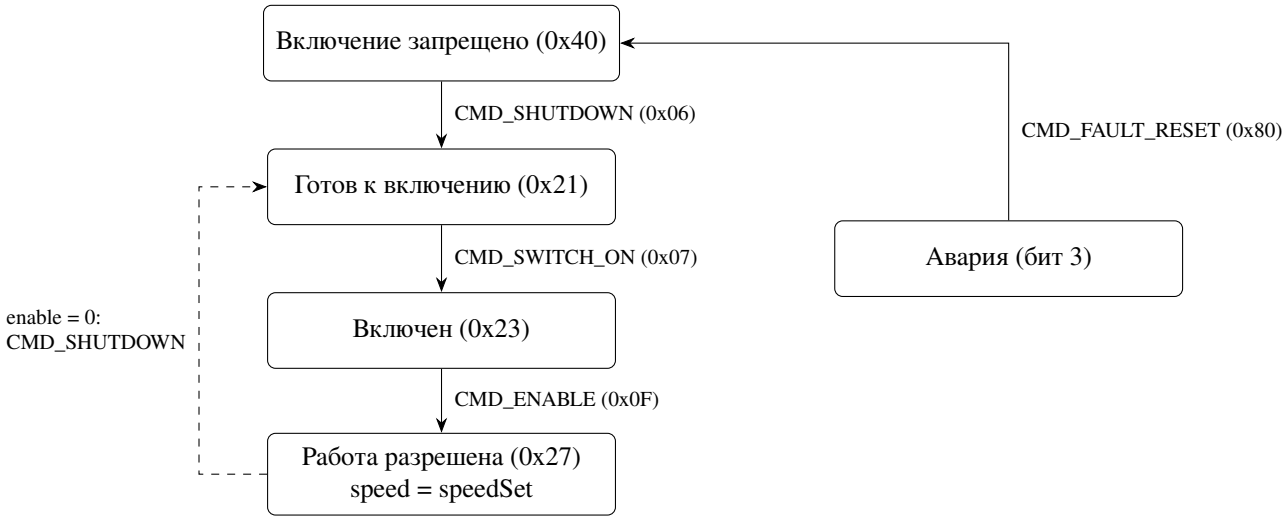


Рис. 4 — Схема управления частотным преобразователем

Имена констант `STATE_SWITCHED_ON` и `STATE_READY` взяты из схемы методических указаний; по терминологии стандарта CiA 402 им соответствуют состояния «Ready to switch on» и «Switched on».

Следующим был разработан функциональный блок `DoorStateMashine`, реализующий основную логику программы. Блок определяет положение двери и задает направление и скорость движения в зависимости от положения двери

Таблица 7 — Коды состояний и команд частотного преобразователя

Константа	**Значение**	**Описание**
DRV_STATE_MASK	0x006F	маска битов состояния
STATE_SWITCHED_ON	0x0021	готов к включению
STATE_READY	0x0023	включен
STATE_ENABLED	0x0027	работа разрешена
STATE_FAULT_BIT	0x0008	бит аварии
CMD_SHUTDOWN	0x0006	команда «выключение»
CMD_SWITCH_ON	0x0007	команда «включить»
CMD_ENABLE	0x000F	команда «разрешить работу»
CMD_FAULT_RESET	0x0080	команда «сброс ошибки»

и требуемого направления. Входы и выходы блока приведены в таблице 8.

Таблица 8 — Параметры функционального блока DoorStateMashine

Конфигурация	**Имя**	**Тип**	**Описание**
вход	state	UINT	слово состояния частотного преобразователя
вход	sw1	BOOL	сигнал датчика S_0 (дверь открыта)
вход	sw2	BOOL	сигнал датчика S_1
вход	sw3	BOOL	сигнал датчика S_2
вход	sw4	BOOL	сигнал датчика S_3 (дверь закрыта)
вход	direction	BOOL	положение тумблера: 1 — закрыть, 0 — открыть
выход	speed	INT	заданная скорость
выход	doorState	DoorStates	текущее состояние двери
внутреннее состояние	s0Prev... s3Prev	BOOL	уровни датчиков в предыдущем цикле

Конечный автомат реализован с помощью оператора switch по переменной doorState. В начале каждого цикла блок сравнивает текущие уровни датчиков с уровнями предыдущего цикла и формирует признаки изменения уровня s0Changed... s3Changed. Затем проверяется готовность преобразователя: если он не находится в состоянии «работа разрешена», автомат переходит в ST_INIT.

После этого в ветви switch, соответствующей текущему состоянию, задается скорость и проверяются условия перехода, приведенные в таблице 4. Тумблер проверяется раньше датчика, поэтому при одновременном срабатывании приоритет имеет реверс.

В конце цикла текущие уровни датчиков сохраняются для следующего цикла. Пока автомат находится в состоянии ST_INIT, сохраненные уровни принудительно обнуляются. Если при включении дверь уже стоит на крайнем датчике, при переходе в ST_UNKNOWN это воспринимается как изменение уровня $0 \to 1$, и автомат сразу определяет положение двери.

Третий функциональный блок LedStateMashine управляет светодиодными индикаторами. Входы и выходы блока приведены в таблице 9.

Таблица 9 — Параметры функционального блока LedStateMashine

Конфигурация	**Имя**	**Тип**	**Описание**
вход	state	UINT	состояние двери
вход	cycleTime	UDINT	время цикла задачи [мс]
выход	led1...led4	BOOL	сигналы индикаторов S_0–S_3
выход	timer	INT	время внутри полупериода мигания [мс]
внутреннее состояние	elapsed	UDINT	накопленное время мигания [мс]
внутреннее состояние	blink	BOOL	текущая фаза мигания

Блок сопоставляет каждому состоянию двери код индикации из таблицы 4. Запись выходов выполняется вспомогательной функцией SetLeds, которая присваивает значения сразу четырем выходам; порядок ее аргументов совпадает с порядком цифр в коде индикации. Мигание с частотой 1 Гц в состоянии ST_UNKNOWN реализовано так: в каждом цикле к переменной elapsed прибавляется время цикла задачи cycleTime. Когда накопленное время достигает 500 мс (константа BLINK_HALF_PERIOD_MS), переменная blink инвертируется, а elapsed обнуляется. Таким образом, индикаторы 500 мс горят и 500 мс не горят, что соответствует периоду 1 с. Поскольку время цикла передается на вход блока, частота мигания не зависит от того, в каком классе задач выполняется программа.

После создания всех функциональных блоков была разработана основная

программа `Main.c`. Структурная схема программного обеспечения приведена на рисунке 5. Функциональные блоки объединены в основной программе, которая выполняется в классе задач Cyclic#4. Входные и выходные сигналы связаны с переменными функциональных блоков через мэпинг (таблица 3).

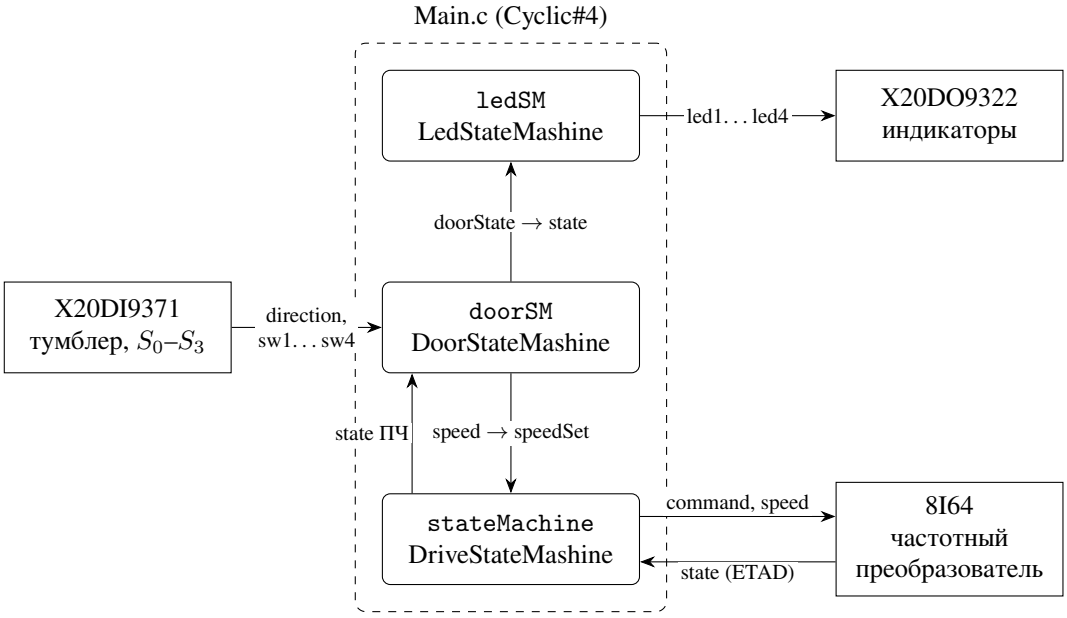


Рис. 5 — Структурная схема программного обеспечения

В подпрограмме инициализации `ProgramInit` с помощью функционального блока `RTInfo` библиотеки `brsystem` определяется время цикла задачи, которое передается на вход `cycleTime` блока индикации. Там же функциональные блоки приводятся в начальное состояние: состояние двери `ST_INIT`, нулевая скорость, погашенные индикаторы.

В основном цикле `ProgramCyclic` функциональные блоки вызываются последовательно:

1) слово состояния преобразователя передается в блок двери (`doorSM.state = stateMachine.state`), после чего вызывается `DoorStateMashine`; сигналы датчиков и тумблера поступают в блок через мэпинг;

2) рассчитанная скорость передается в блок управления преобразователем (`stateMachine.speedSet = doorSM.speed`), его работа разрешается (`enable = 1`), и вызывается `DriveStateMashine`; выходы `command` и `speed` через мэпинг поступают на преобразователь;

3) состояние двери передается в блок индикации (`ledSM.state =`

doorSM.doorState), и вызывается LedStateMashine; выходы led1...led4 через мэпинг поступают на индикаторы.

Логика каждого блока не зависит от остальных: блок двери не содержит сведений о протоколе преобразователя, а блок индикации — о датчиках. Поэтому блоки могут повторно использоваться в других проектах. Все численные параметры (скорости, коды преобразователя, период мигания) вынесены в файл констант библиотеки Constants.var, что позволяет изменять их в одном месте.

Для подтверждения работоспособности системы управления на стенде был проведен эксперимент, представляющий собой полный цикл управления дверью: от подачи питания до отработки команд на открытие и закрытие. С помощью инструмента Trace регистрировались следующие величины:

- заданная скорость вращения приводного двигателя stateMachine.speed;
- положение тумблера doorSM.direction;
- состояния датчиков doorSM.sw1...doorSM.sw4;
- состояния светодиодных индикаторов ledSM.led1...ledSM.led4;
- состояние конечного автомата doorSM.doorState.

Результаты эксперимента приведены на рисунке 6. После подачи питания преобразователь последовательно переводится в состояние «работа разрешена», и автомат переходит в состояние ST_UNKNOWN. Дверь движется в сторону, заданную тумблером, с малой скоростью 300, при этом все индикаторы мигают с частотой 1 Гц. При изменении уровня крайнего датчика автомат определяет положение двери (ST_OPEN или ST_CLOSE) и останавливает двигатель.

При переключении тумблера на закрытие заданная скорость принимает значение +300 на отрезке s_0s_1, +800 на отрезке s_1s_2 и снова +300 на отрезке s_2s_3. Переключение скорости происходит в моменты изменения уровня датчиков S_1, S_2, а при изменении уровня датчика S_3 скорость становится равной нулю. Индикаторы при этом загораются по одному до кода 1111. При переключении тумблера на открытие процесс повторяется в обратном порядке с отрицательной скоростью, индикаторы гаснут до кода 0000. При переключении тумблера во время движения дверь меняет направление на том же отрезке хода.

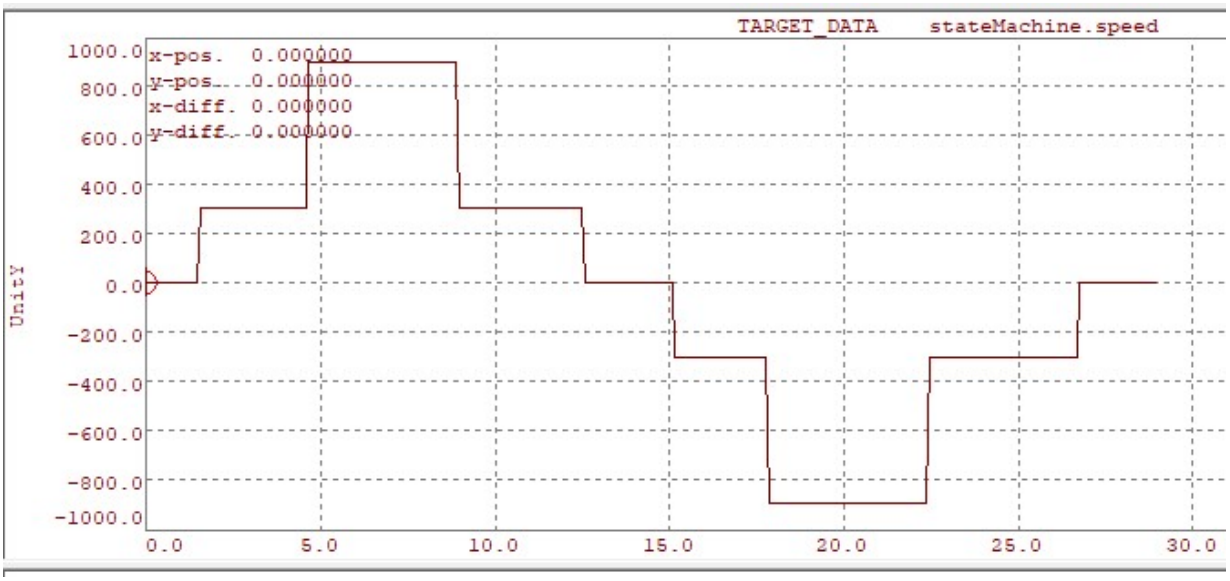


Рис. 6 — Результаты эксперимента (Trace): заданная скорость, состояния датчиков, тумблера и индикаторов

ВЫВОДЫ

В ходе выполнения лабораторной работы было разработано программное обеспечение системы автоматического управления приводом защитной двери в среде Automation Studio на языке Си. Были освоены конфигурирование оборудования, настройка частотного преобразователя по паспортным данным асинхронного двигателя и мэпинг переменных на каналы модулей ввода-вывода.

Программное обеспечение построено из трех функциональных блоков: `DriveStateMashine` включает частотный преобразователь по профилю CiA 402 и передает ему задание скорости; `DoorStateMashine` реализует конечный автомат двери; `LedStateMashine` отображает положение двери на индикаторах. Блоки независимы друг от друга и пригодны для повторного использования. Конечный автомат двери реализован оператором `switch` по перечислению `DoorStates`. Переходы выполняются последовательно по изменению логического уровня датчиков, что исключает пропуск состояний и обеспечивает корректную смену скорости при реверсе.

Эксперимент подтвердил работоспособность системы. При включении дверь на малой скорости определяет свое положение, при этом индикаторы мигают с частотой 1 Гц. В рабочем режиме движение на отрезке $s_1 s_2$ выполняется на максимальной скорости, а на отрезках $s_0 s_1$ и $s_2 s_3$ — на малой. Индикаторы показывают местоположение двери.

Таким образом, цель работы достигнута: получены навыки построения

программного обеспечения промышленных систем управления на базе функциональных блоков и конечных автоматов.

Приложение А

Листинг исходного программного кода файла DoorStateMashine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
	#include "Library.h"
#ifdef __cplusplus
	};
#endif

void DoorStateMashine(struct DoorStateMashine* inst)
{
	BOOL s0 = inst->sw1;
	BOOL s1 = inst->sw2;
	BOOL s2 = inst->sw3;
	BOOL s3 = inst->sw4;

	BOOL s0Changed = (s0 != inst->s0Prev);
	BOOL s1Changed = (s1 != inst->s1Prev);
	BOOL s2Changed = (s2 != inst->s2Prev);
	BOOL s3Changed = (s3 != inst->s3Prev);

	BOOL driveOn = ((inst->state & DRV_STATE_MASK) == STATE_ENABLED);
	DoorStates stateAtStart;

	if (!driveOn && inst->doorState != ST_INIT)
		inst->doorState = ST_INIT;

	stateAtStart = inst->doorState;

	switch (inst->doorState)
	{
		case ST_INIT:
			inst->speed = SPEED_STOP;
			if (driveOn)
				inst->doorState = ST_UNKNOWN;
			break;

		case ST_UNKNOWN:
			inst->speed = inst->direction ? SPEED_SLOW :
-SPEED_SLOW;
			if (!inst->direction && s0Changed)
```

```
            inst->doorState = ST_OPEN;
      else if (inst->direction && s3Changed)
            inst->doorState = ST_CLOSE;
      break;

case ST_OPEN:
      inst->speed = SPEED_STOP;
      if (inst->direction)
            inst->doorState = ST_ACC_POS;
      break;

case ST_ACC_POS:
      inst->speed = SPEED_SLOW;
      if (!inst->direction)
            inst->doorState = ST_DEC_NEG;
      else if (s1Changed)
            inst->doorState = ST_POS;
      break;

case ST_POS:
      inst->speed = SPEED_FAST;
      if (!inst->direction)
            inst->doorState = ST_NEG;
      else if (s2Changed)
            inst->doorState = ST_DEC_POS;
      break;

case ST_DEC_POS:
      inst->speed = SPEED_SLOW;
      if (!inst->direction)
            inst->doorState = ST_ACC_NEG;
      else if (s3Changed)
            inst->doorState = ST_CLOSE;
      break;

case ST_CLOSE:
      inst->speed = SPEED_STOP;
      if (!inst->direction)
            inst->doorState = ST_ACC_NEG;
      break;

case ST_ACC_NEG:
      inst->speed = -SPEED_SLOW;
      if (inst->direction)
            inst->doorState = ST_DEC_POS;
      else if (s2Changed)
            inst->doorState = ST_NEG;
```

```
                break;

        case ST_NEG:
                inst->speed = -SPEED_FAST;
                if (inst->direction)
                        inst->doorState = ST_POS;
                else if (s1Changed)
                        inst->doorState = ST_DEC_NEG;
                break;

        case ST_DEC_NEG:
                inst->speed = -SPEED_SLOW;
                if (inst->direction)
                        inst->doorState = ST_ACC_POS;
                else if (s0Changed)
                        inst->doorState = ST_OPEN;
                break;

        default:
                inst->speed = SPEED_STOP;
                inst->doorState = ST_INIT;
                break;
    }

    if (stateAtStart == ST_INIT || inst->doorState == ST_INIT)
    {
        inst->s0Prev = 0;
        inst->s1Prev = 0;
        inst->s2Prev = 0;
        inst->s3Prev = 0;
    }
    else
    {
        inst->s0Prev = s0;
        inst->s1Prev = s1;
        inst->s2Prev = s2;
        inst->s3Prev = s3;
    }
}
```

Приложение Б

Листинг исходного программного кода файла DriveStateMashine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
#include "Library.h"
#ifdef __cplusplus
	};
#endif

void DriveStateMashine(struct DriveStateMashine* inst)
{
	UINT st = inst->state & DRV_STATE_MASK;

	if (!inst->enable)
	{
		inst->command = CMD_SHUTDOWN;
		inst->speed   = 0;
		return;
	}

	if (inst->state & STATE_FAULT_BIT)
	{
		inst->command = CMD_FAULT_RESET;
		inst->speed   = 0;
		return;
	}

	if (st == STATE_ENABLED)
	{
		inst->command = CMD_ENABLE;
		inst->speed = inst->speedSet;
	}
	else if (st == STATE_READY)
	{
		inst->command = CMD_ENABLE;
		inst->speed = 0;
	}
	else if (st == STATE_SWITCHED_ON)
	{
		inst->command = CMD_SWITCH_ON;
		inst->speed   = 0;
```

```
        }
        else
        {
                inst->command = CMD_SHUTDOWN;
                inst->speed   = 0;
        }
}
```

Приложение В

Листинг исходного программного кода файла LedStateMashine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
	#include "Library.h"
#ifdef __cplusplus
	};
#endif

static void SetLeds(struct LedStateMashine* inst, BOOL l1, BOOL l2, BOOL
l3, BOOL l4)
{
	inst->led1 = l1;
	inst->led2 = l2;
	inst->led3 = l3;
	inst->led4 = l4;
}

void LedStateMashine(struct LedStateMashine* inst)
{
	switch (inst->state)
	{
		case ST_UNKNOWN:
			inst->elapsed += dt;
			if (inst->elapsed >= BLINK_HALF_PERIOD_MS)
			{
				inst->elapsed = 0;
				inst->blink = !inst->blink;
			}
			SetLeds(inst, inst->blink, inst->blink, inst->blink,
inst->blink);
			break;

		case ST_OPEN:	SetLeds(inst, 0, 0, 0, 0); break;
		case ST_ACC_POS:	SetLeds(inst, 1, 0, 0, 0); break;
		case ST_POS:	SetLeds(inst, 1, 1, 0, 0); break;
		case ST_DEC_POS:	SetLeds(inst, 1, 1, 1, 0); break;
		case ST_CLOSE:	SetLeds(inst, 1, 1, 1, 1); break;
		case ST_ACC_NEG:	SetLeds(inst, 1, 1, 1, 0); break;
		case ST_NEG:	SetLeds(inst, 1, 1, 0, 0); break;
		case ST_DEC_NEG:	SetLeds(inst, 1, 0, 0, 0); break;
```

```
		case ST_INIT:
		default:
			SetLeds(inst, 0, 0, 0, 0);
			break;
	}

	if (inst->state != ST_UNKNOWN)
	{
		inst->elapsed = 0;
		inst->blink = 0;
	}
	inst->timer = (INT)inst->elapsed;
}
```

Приложение Г

Листинг исходного программного кода файла Main.c

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
	#include <AsDefault.h>
#endif

void _INIT ProgramInit(void)
{
	stateMachine.enable   = 0;
	stateMachine.speedSet = 0;
	stateMachine.command  = CMD_SHUTDOWN;
	stateMachine.speed    = 0;

	doorSM.doorState = ST_INIT;
	doorSM.speed     = 0;

	ledSM.state = ST_INIT;
	ledSM.led1  = 0;
	ledSM.led2  = 0;
	ledSM.led3  = 0;
	ledSM.led4  = 0;
}


void _CYCLIC ProgramCyclic(void)
{
	doorSM.state = stateMachine.state;
	DoorStateMashine(&doorSM);

	stateMachine.enable   = 1;
	stateMachine.speedSet = doorSM.speed;
	DriveStateMashine(&stateMachine);

	ledSM.state = (UINT)doorSM.doorState;
	LedStateMashine(&ledSM);
}


void _EXIT ProgramExit(void)
{

}
```